

Referral Staking

Smart Contract Security Assessment

May 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Self Transfer Can Clear User Balances Without Moving Tokens	6
4.2	Permissionless Fee Claims Can Fragment Penalty Fees	7
4.3	Permissionless Stake Can Disrupt User Escrow Operations	7
4.4	Missing Minimum Stake Check in cancel_unstake	8
4.5	Informational and Undetermined Issues	10
5	Disclaimer	12



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Referral Staking* smart contracts, starting on May 13th, 2026, and concluding on May 18th, 2026.

Project Overview

Meteora Referral Staking lets users stake supported SPL tokens into a program controlled vault through per user escrow accounts. It supports standard staking, cooldown based unstaking, immediate unstaking with configurable penalty fees, withdrawal after the cooldown period, and penalty fee collection by a configured fee receiver. It also includes owner controlled administration for updating staking parameters, changing the fee receiver, and transferring staking ownership.

Audit Scope

The assessment scope contains mainly the smart contracts of the referral program for the *Referral Staking* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Referral Staking
 - Codebase: <https://github.com/MeteoraAg/referral-staking>
 - Commit Hash: 29bfc910ac23174edc5672e9b97f7f81aab6d097

The issues described in this report have been resolved by the following commit:

- Referral Staking
 - Codebase: <https://github.com/MeteoraAg/referral-staking>
 - Commit Hash: d2db93aabeca4404b7c75b635fe160b43e981c37

We listed the files we have audited below:

- Referral Staking
 - programs/referral/src/access_control.rs
 - programs/referral/src/const_pda.rs
 - programs/referral/src/constants.rs
 - programs/referral/src/errors.rs
 - programs/referral/src/events.rs
 - programs/referral/src/instructions/mod.rs
 - programs/referral/src/instructions/permissioned/ix_transfer_owner.rs
 - programs/referral/src/instructions/permissioned/ix_update_config.rs
 - programs/referral/src/instructions/permissioned/ix_update_penalty_fee_receiver.rs
 - programs/referral/src/instructions/permissioned/mod.rs



- programs/referral/src/instructions/permissionless/ix_cancel_unstake.rs
- programs/referral/src/instructions/permissionless/ix_claim_penalty_fee.rs
- programs/referral/src/instructions/permissionless/ix_close_escrow.rs
- programs/referral/src/instructions/permissionless/ix_immediate_unstake.rs
- programs/referral/src/instructions/permissionless/ix_initialize_escrow.rs
- programs/referral/src/instructions/permissionless/ix_initialize_staking.rs
- programs/referral/src/instructions/permissionless/ix_stake.rs
- programs/referral/src/instructions/permissionless/ix_unstake.rs
- programs/referral/src/instructions/permissionless/ix_withdraw.rs
- programs/referral/src/instructions/permissionless/mod.rs
- programs/referral/src/lib.rs
- programs/referral/src/math/fee.rs
- programs/referral/src/math/mod.rs
- programs/referral/src/math/safe_math.rs
- programs/referral/src/states/escrow.rs
- programs/referral/src/states/mod.rs
- programs/referral/src/states/staking.rs
- programs/referral/src/states/unstake.rs
- programs/referral/src/utils/mod.rs
- programs/referral/src/utils/token.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 4 low issues
- 3 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Self Transfer Can Clear User Balances Without Moving Tokens	Low	Fixed
02	Permissionless Fee Claims Can Fragment Penalty Fees	Low	Fixed
03	Permissionless Stake Can Disrupt User Escrow Operations	Low	Fixed
04	Missing Minimum Stake Check in cancel_unstake	Low	Acknowledged
05	Possible Incorrect Token Account Fields in Transfer Events	Informational	Fixed
06	Saturating Metrics Updates May Lead to Accounting Inconsistencies	Informational	Fixed
07	Fee Receiver Updates Can Reassign Unclaimed Penalty Fees	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Self Transfer Can Clear User Balances Without Moving Tokens

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The `withdraw` and `immediate_unstake` IXs only validate that `owner_token_vault` uses the expected mint:

```
40     #[account(  
41         mut,  
42         token::mint = token_mint,  
43         token::token_program = token_program,  
44     )]  
45     pub owner_token_vault: Box<Account<'info, TokenAccount>>,
```

[programs/referral/src/instructions/permissionless/ix_withdraw.rs#L40-L45](#)

However, they do not ensure that `owner_token_vault` is different from `token_vault`. If both accounts are the same, the token transfer becomes a self transfer from the staking vault to itself. SPL Token accepts this transfer, but no tokens are moved, while the program still decreases the user's internal staking or unstaking balance.

Impact

A user can accidentally clear their staking or unstaking balance without receiving tokens, leaving the funds as untracked surplus in the staking vault.

Recommendation

It's recommended to add an explicit account check in `withdraw` and `immediate_unstake`, or enforce it directly in `transfer_from_pool_to_user`, to ensure the destination token account is not the staking `token_vault`:

```
owner_token_vault.key() != token_vault.key()
```

Mitigation Review Log

Fixed in the commit `8cba3806f0dde7cdbe3d8d00e4b2c1c33da1ee7a`.



4.2 Permissionless Fee Claims Can Fragment Penalty Fees

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The `claim_penalty_fee` IX is permissionless and only requires `fee_receiver_token_account.owner` to equal the `stakingConfig.fee_receiver`. Anyone can create multiple token accounts owned by the fee receiver and claim penalty fees into those accounts.

```
41     #[account(  
42         mut,  
43         constraint = fee_receiver_token_account.owner ==  
44         ↪ staking.load()?.config.fee_receiver @  
45         ↪ ReferralError::InvalidFeeReceiver,  
46         token::mint = token_mint.key(),  
47         token::token_program = token_program.key(),  
48     )]  
49     pub fee_receiver_token_account: Box<Account<'info, TokenAccount>>,  
50     programs/referral/src/instructions/permissionless/ix_claim_penalty_  
51     fee.rs#L41-L47
```

Impact

This can fragment the fee receiver's funds across many token accounts and create extra operational overhead for discovery and consolidation.

Recommendation

It is recommended to enforce the fee receiver's ATA as the claim destination.

Mitigation Review Log

Fixed in the commit b02c3449c5e9697f185efde20a6d5874d0d9e6b1.

4.3 Permissionless Stake Can Disrupt User Escrow Operations

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation



Description

The `stake` IX allows anyone to stake into an existing escrow without authorization from the escrow owner. As a result, a third party can change a user's `escrow.staked_amount` before the user performs escrow lifecycle operations such as `close_escrow` or a `full unstake`.

For example, if a user tries to close an empty escrow, a third party can stake into it before the transaction lands, causing `close_escrow` to fail because the escrow is no longer empty. Similarly, if a frontend submits the user's exact staked amount as `max_amount` for a full unstake, a third party can stake into the escrow first, causing the transaction to no longer fully exit the position and potentially fail due to the minimum stake check.

Impact

This can cause failed transactions or extra steps for users. In the worst case, a user may need to repeatedly resubmit transactions if a third party keeps changing the escrow balance before the user's transaction lands.

Recommendation

It is recommended to require the escrow owner to authorize stake, so a third party cannot change a user's escrow state without the owner's approval.

Mitigation Review Log

Fixed in the commit `edea525f2021f15158a803aa3443f25a3e599dc6`.

4.4 Missing Minimum Stake Check in `cancel_unstake`

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

Description

The program enforces the minimum stake invariant in `stake`, `unstake` and `immediate_unstake`.



```
193     pub fn validate_escrow_minimum_stake_amount(&self, escrow: &Escrow) ->
194         Result<()> {
195         require!(
196             escrow.staked_amount >= self.min_staking_amount ||
197             escrow.staked_amount == 0,
198             ReferralError::BelowMinimumStakeAmount
199         );
200         Ok(())
201     }
```

[programs/referral/src/states/staking.rs#L193-L199](#)

However, `cancel_unstake` moves tokens from `partial_unstaked_amount` back to `staked_amount` without running the same invariant check.

```
76     pub fn cancel_unstake(&mut self, escrow: &mut Escrow, amount: u64) ->
77         Result<()> {
78         escrow.cancel_unstake(amount)?;
79
80         self.staked_amount = self.staked_amount.safe_add(amount)?;
81         self.partial_unstaked_amount =
82             self.partial_unstaked_amount.safe_sub(amount)?;
83         Ok(())
84     }
```

[programs/referral/src/states/staking.rs#L76-L82](#)

This allows an escrow to first reach a valid zero-stake state, and then restore a small cancelled unstake amount into `staked_amount`, resulting in a non-zero balance below `min_staking_amount`.

Impact

The escrow can end up with a non-zero staked balance below the configured minimum stake, bypassing the intended minimum stake invariant.

Recommendation

It is recommended to add `validate_escrow_minimum_stake_amount` after `escrow.cancel_unstake(amount)` in `Staking::cancel_unstake`.

Mitigation Review Log

Meteora Team: It's a design choice. We add docs on the code.



```
// Note: We allow cancel unstake even if after cancelled, the staking
↳ amount is still less than the minimum staking amount. This is to
↳ allow users to cancel unstake in case they want to add more stake
↳ later to reach the minimum staking amount.
// Eg: User can cancel multiple unstake until the total staking
↳ amount is greater than or equal to the minimum staking amount.
```

4.5 Informational and Undetermined Issues

Possible Incorrect Token Account Fields in Transfer Events

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In `ImmediateUnstakeEvt`, `WithdrawEvt`, and `ClaimPenaltyFeeEvt`, the event comments describe the receiver field as the token account that receives the tokens.

```
185 /// The account that receives the unstake token
186 pub receiver: Pubkey,
```

[programs/referral/src/events.rs#L185-L186](#)

However, the emitted value is the owner of the token account instead of the token account itself, such as `owner_token_vault.owner` and `fee_receiver_token_account.owner`. If off-chain parts rely on these event values, they may not be able to directly identify the token account used in the transfer from the owner address alone.

It is recommended to keep the emitted values consistent with the event comments by emitting the token account address, or emit both the token account and its owner if both values are needed.

Fixed in the commit `39f88bb2164ca834efd6ead747d11a429ed611e2`.

Saturating Metrics Updates May Lead to Accounting Inconsistencies

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

`StakingMetrics::increment_total_escrow` and `StakingMetrics::decrement_total_escrow` update `total_escrow_count` with `saturating_add(1)` and `saturating_sub(1)`. If an overflow or underflow occurs, these operations will not return an error and will silently keep the value at the numeric limit instead.

Although such an overflow or underflow is theoretically unlikely in practice, it is recommended to use the existing `safe_add` and `safe_sub` helpers to fully avoid silently masking this type of accounting issue.



Fixed in the commit 0567a94858eadfab1beed2bf7950782541b2e302.

Fee Receiver Updates Can Reassign Unclaimed Penalty Fees

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

`update_penalty_fee_receiver` changes `staking.config.fee_receiver`, while accrued penalty fees are stored only in the global `staking.penalty_fee_amount`. Therefore, if fees accrue under fee receiver A and the owner changes the receiver to B before claiming, B can claim the previously accrued fees.

- This is fine if the fee receiver is only an owner-controlled payout address.
- If fee receivers are separate recipients, it is recommended to require `staking.penalty_fee_amount == 0` before allowing the fee receiver to be updated.

Meteora Team: It's design choice where fee receiver is only an owner-controller payout address.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

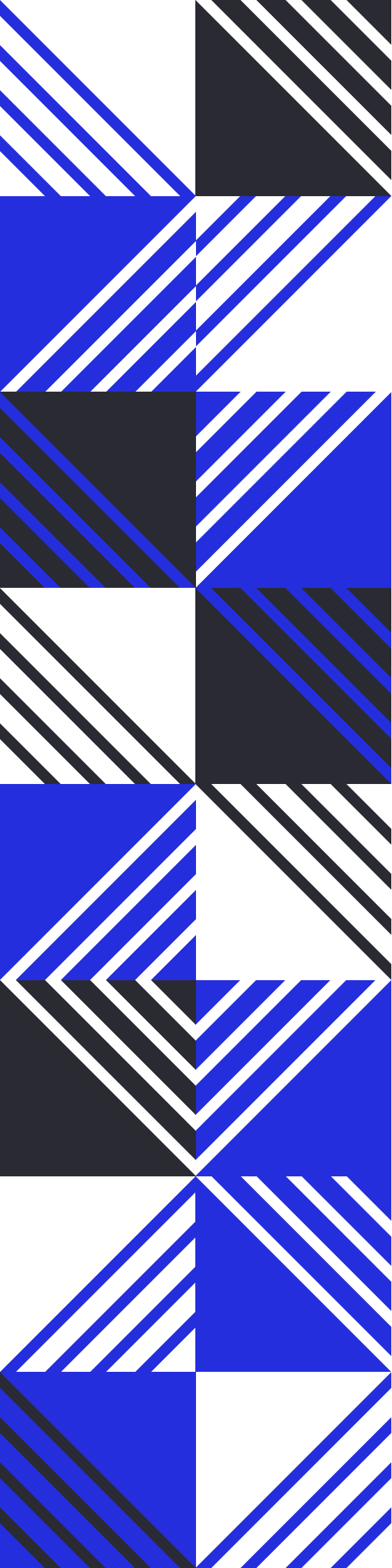
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs